

Repository & Pipeline Structure

Full folder and file layout — common layer, 15 market-gate phases, configuration, tests, and runtime artifacts.

Alqueva-PSP-PV-BESS-24hr-Energy-Trading-DA-IDA-aFRR-mFRR-Optimizer/

— run_production.py	# ◀ Master orchestrator — all 15 phases
— common_layer/	# Shared foundation — imported by every phase
— configuration/	
— config_loader.py	# Load_config() → AppConfig
— market_config.py	# MarketConfig dataclass
— plant_config.py	# PlantConfig dataclass
— solver_config.py	# SolverConfig dataclass
— optimisation_model/	
— core_milp_builder.py	# CoreModelMeta, build_milp() — ONE model for all gates
— core_milp_solver.py	# solve_milp(), SolveError — CPLEX → HiGHS → CBC
— ida_reoptimiser.py	# optimise_ida() — freeze hours, re-solve
— reserve_offer_builder.py	# build_afrr_offers(), build_mfrr_offers()
— activation_ramp_tracker.py	# ramp-corrected eff_isp_h for settlement
— reserve_activation.py	# reserve activation helpers
— physical_plant_models/	
— psp_turbine_pump_model.py	# PSPModel, UnitDispatch (4 Francis units)
— pv_production_model.py	# PVModel (5 MWp, temperature derate, degradation)
— bess_model.py	# BESSModel, BESSDispatch (1 MW / 2 MWh)
— reservoir_model.py	# ReservoirModel (Alqueva + Pedrógão)
— fcr_headroom_model.py	# FCRHeadroomModel (reserved — never sold)
— reservoir_activation_checker.py	# validates long-pumping sequences
— database/	
— position_store.py	# PositionStore → runtime/db/positions.db
— reserve_store.py	# ReserveStore → runtime/db/reserve.db
— realtime_store.py	# DeliveryStore, ActivationStore → realtime.db
— component_store.py	# ComponentStore → runtime/components/<date>.json
— audit_store.py	# AuditStore (read-only) → audit_<date>.jsonl
— schema_validator.py	# input schema validation
— gate_scheduler/	
— gate_scheduler.py	# GateScheduler — CET gate-time resolver
— gate_trigger_spec.py	# trigger definitions from market.yaml
— utilities/	
— audit_logger.py	# AuditLogger — append-only JSONL trail
— timezone_utils.py	# CET (OMIE) ↔ WET/CET (plant) conversions
— date_utils.py	# delivery date parsing, D-1 calculations
— logging_utils.py	# phase-prefixed logger setup
— phase_1_da_day_ahead_bidding/	# — Phase 1, Day-Ahead —————

```

├── run_da.py                                # entry point
├── da_price_pv_inflow_forecasting/
│   ├── da_price_forecaster.py              # ML DA price forecast
│   ├── da_price_train_val_test.py
│   ├── pv_power_forecaster.py              # PV production forecast
│   ├── pv_train_val_test.py
│   ├── reservoir_inflow_forecaster.py      # natural inflow forecast
│   ├── reservoir_inflow_train_val_test.py
│   ├── omie_da_price_loader.py             # Live OMIE DA price loader
│   └── ml_train_val_test_common.py         # shared ML utilities
├── da_bid_formatting/
│   ├── da_bid_formatter.py                 # formats OMIE bid payload
│   └── da_bid_checker.py                  # physical bid validation
├── risk_and_bid_validation/
│   └── pre_trade_risk_checker.py           # pre-trade risk limits
├── trader_approval/
│   └── trader_approval_prompt.py           # [A]/[R] interactive prompt
├── phase_2a_ida1_intraday_auction_1/      # — Phase 2A, IDA1 —
│   ├── run_ida1.py                         # entry point
│   ├── ida1_price_forecasting/
│   │   ├── ida1_price_forecaster.py
│   │   ├── ida1_price_train_val_test.py
│   │   └── omie_ida1_price_loader.py
│   ├── ida1_milp_reoptimiser/
│   │   └── ida1_reoptimiser.py             # freeze DA, re-solve H1-H24
│   └── ida1_bid_formatting/
│       └── ida1_bid_formatter.py           # SIDC delta bid payload
├── phase_2b_ida2_intraday_auction_2/      # — Phase 2B, IDA2 —
│   ├── run_ida2.py
│   ├── ida2_price_forecasting/
│   │   ├── ida2_price_forecaster.py
│   │   ├── ida2_price_train_val_test.py
│   │   └── omie_ida2_price_loader.py
│   ├── ida2_milp_reoptimiser/
│   │   └── ida2_reoptimiser.py             # freeze H1-H2, re-solve H3-H24
│   └── ida2_bid_formatting/
│       └── ida2_bid_formatter.py
├── phase_2c_ida3_intraday_auction_3/      # — Phase 2C, IDA3 —
│   ├── run_ida3.py
│   ├── ida3_price_forecasting/
│   │   ├── ida3_price_forecaster.py
│   │   ├── ida3_price_train_val_test.py
│   │   └── omie_ida3_price_loader.py
│   ├── ida3_milp_reoptimiser/
│   │   └── ida3_reoptimiser.py             # freeze H1-H11, re-solve H12-H24
│   └── ida3_bid_formatting/
│       └── ida3_bid_formatter.py

```

```

phase_2d_xbid_continuous_intraday/      # — Phase 2D, XBID —————
├── run_xbid.py
├── xbid_price_forecasting/
│   ├── xbid_price_forecaster.py
│   ├── xbid_price_train_val_test.py
│   ├── xbid_price_loader.py
│   └── create_xbid_training_data.py
├── xbid_milp_optimiser/
│   └── xbid_optimiser.py                # per-order caps, H-1 rolling
├── xbid_bid_formatting/
│   └── xbid_bid_formatter.py
└──

phase_3a_afrr_automatic_frequency_reserve/  # — Phase 3A, aFRR —————
├── run_afrr.py
├── afrr_price_forecasting/
│   ├── afrr_price_forecaster.py
│   ├── afrr_price_train_val_test.py
│   ├── picasso_afrr_price_loader.py    # PICASSO Live price Loader
│   └── create_afrr_training_data.py
├── afrr_reserve_offer_builder/
│   ├── afrr_offer_builder.py           # headroom → symmetric up/dn offers
│   └── afrr_offer_checker.py           # FAT deliverability, cap ≤ 250 EUR/MW
└──

phase_3b_mfrr_manual_frequency_reserve/     # — Phase 3B, mFRR —————
├── run_mfrr.py
├── mfrr_price_forecasting/
│   ├── mfrr_price_forecaster.py
│   ├── mfrr_price_train_val_test.py
│   ├── mari_mfrr_price_loader.py       # MARI Live price Loader
│   └── create_mfrr_training_data.py
├── mfrr_reserve_offer_builder/
│   ├── mfrr_offer_builder.py
│   └── mfrr_offer_checker.py
└──

phase_4a_isp_real_time_dispatch/            # — Phase 4A, Real-Time —————
├── run_realtime.py
├── isp_setpoint_dispatch/
│   ├── psp_setpoint_dispatcher.py      # PSP unit setpoints per ISP
│   └── bess_setpoint_dispatcher.py    # BESS setpoints per ISP
├── isp_activation_tracking/
│   └── isp_position_tracker.py         # scheduled vs actual per ISP
├── telemetry/
│   └── ren_isp_signal_loader.py        # REN telemetry feed
└──

phase_4b_afrr_activation_response/          # — Phase 4B, aFRR Activation —————
├── run_afrr_activation.py
├── afrr_setpoint_dispatch/
│   └── afrr_activation_handler.py      # ramp, min hold 2 ISPs, eff_isp_h
├── afrr_activation_tracking/
└──

```

└─ afrr_activation_logger.py	
phase_4c_mfrr_activation_response/	# — Phase 4C, mFRR Activation —————
└─ run_mfrr_activation.py	
└─ mfrr_setpoint_dispatch/	
└─ mfrr_activation_handler.py	# ramp, min hold 3 ISPs, eff_isp_h
└─ mfrr_activation_tracking/	
└─ mfrr_activation_logger.py	
phase_5a_da_ida_settlement/	# — Phase 5A, Energy Settlement —————
└─ run_energy_settlement.py	
└─ energy_settlement_calculation/	
└─ da_settlement_calculator.py	# DA volume × OMIE settlement price
└─ ida_settlement_calculator.py	# IDA delta per gate, no double-counting
└─ omie_settlement_price_loader.py	# final OMIE settlement prices
phase_5b_reserve_settlement/	# — Phase 5B, Reserve Settlement —————
└─ run_reserve_settlement.py	
└─ reserve_settlement_calculation/	
└─ afrr_settlement_calculator.py	# capacity (hourly) + activation, eff_isp_h
└─ mfrr_settlement_calculator.py	# reuses generic settle_reserve()
└─ ren_reserve_settlement_loader.py	# PICASSO + MARI invoice loader
phase_5c_imbalance_settlement/	# — Phase 5C, Imbalance Settlement —————
└─ run_imbalance_settlement.py	
└─ imbalance_price_and_volume/	
└─ imbalance_volume_calculator.py	# actual - scheduled per ISP
└─ ren_imbalance_price_loader.py	# REN post-delivery prices
└─ imbalance_settlement_calculation/	
└─ imbalance_settlement_calculator.py	# Long→DA×0.85, short→DA×1.20
phase_5d_analytics_and_reporting/	# — Phase 5D, Analytics —————
└─ run_analytics.py	
└─ analytics_and_kpis/	
└─ daily_pnl_calculator.py	# compute_daily_pnl()
└─ revenue_breakdown_analyzer.py	# revenue_shares() by market stream
└─ kpi_reporter.py	# compute_kpis() - 10 KPI sections
└─ operational_analytics.py	
└─ daily_excel_reports/	
└─ daily_report_exporter.py	# export_daily_report() → .xlsx
└─ dispatch_sheet_builder.py	# Dispatch_Hourly (94 cols)
└─ summary_kpi_builder.py	# Summary_KPIs sheet
phase_6_backtesting_and_validation/	# — Phase 6, Backtesting —————
└─ run_backtest.py	
└─ backtest_engine/	
└─ backtest_runner.py	# historical date-range replay
└─ historical_data_loader.py	# Load historical prices / inflows
└─ forecast_and_model_validation/	
└─ price_forecast_validator.py	# DA / IDA price forecast accuracy

├── pv_forecast_validator.py	# PV production forecast accuracy
├── milp_solution_quality_checker.py	# MIP gap, feasibility, solve time
├── risk_analytics/	
│ ├── portfolio_risk_metrics.py	# VaR, CVaR, revenue volatility
├── backtest_excel_reports/	
│ └── backtest_report_exporter.py	
├── figures/	# — 9 Production Figures —————
│ └── __init__.py	# generate(date) → all 9 figures at 600 DPI
├── config/	# — Configuration —————
│ ├── market.yaml	# gate times, IDA dates, ISP, FAT, bid limits
│ ├── plant.yaml	# PSP / PV / BESS / reservoir, head model
│ ├── solver.yaml	# CPLEX → HiGHS → CBC, MIP gap, time limits
│ └── run.yaml	# date, mode, data source, phase flags
├── tests/	# — Test Suite (pytest) —————
│ ├── conftest.py	# shared fixtures
│ ├── test_e2e_chain.py	# end-to-end pipeline chain
│ ├── test_milp_physics.py	# MILP constraint physics
│ ├── test_ida_frozen.py	# IDA hour-freezing logic
│ ├── test_settlement.py	# settlement calculations
│ ├── test_reserve_checker.py	# reserve offer validation
│ ├── test_reserve_market_deep.py	# deep reserve market tests
│ ├── test_reserve_hidden_invariants.py	# PR-11 no-double-selling invariants
│ ├── test_reserve_realtime_delivery.py	# activation + delivery chain
│ ├── test_checker_negative.py	# negative / edge-case physical checks
│ └── test_bug_regressions.py	# regression suite
├── runtime/	# — Auto-created at first run —————
│ ├── db/	
│ │ ├── positions.db	# SQLite – energy positions (DA, IDA, XBID)
│ │ ├── reserve.db	# SQLite – aFRR / mFRR capacity offers
│ │ └── realtime.db	# SQLite – per-ISP delivery & activations
│ ├── audit/	
│ │ └── audit_YYYY-MM-DD.jsonl	# append-only audit trail (one JSON per event)
│ ├── components/	
│ │ └── components_YYYY-MM-DD.json	# per-component DA dispatch results
│ ├── reports/	
│ │ └── daily_report_YYYY-MM-DD.xlsx	# 5-sheet Excel report
├── docs/	# — Architecture Diagrams —————
│ ├── pipeline_architecture.png	# pipeline diagram (2× retina, 940 px)
│ └── pipeline_architecture.svg	# same diagram as scalable SVG
└── requirements.txt	# pip install -r requirements.txt